

Priority of heap/stack memory:

```

graph TD
    Root[Root] --> L[Left]
    Root --> R[Right]
    L --> LL[Left-Left]
    L --> LR[Left-Right]
    R --> RL[Right-Left]
    R --> RR[Right-Right]
    LL --> LLL[Left-Left-Left]
    LL --> LLR[Left-Left-Right]
    LR --> LRL[Left-Right-Left]
    LR --> LRR[Left-Right-Right]
    RL --> RLL[Right-Left-Left]
    RL --> RLR[Right-Left-Right]
    RR --> RRL[Right-Right-Left]
    RR --> RRR[Right-Right-Right]
  
```

What is a binary tree?

A binary tree is a tree in which each node has at most two children, the left child and the right child. Binary trees are a fundamental data structure in computer science, used for storing and organizing data in a hierarchical manner. They are also used in algorithms for searching, sorting, and traversing data.

What is a binary search tree?

A binary search tree (BST) is a binary tree where each node contains a key, and the left child of a node contains a key less than the node's key, and the right child contains a key greater than the node's key. This structure allows for efficient searching, insertion, and deletion of elements.

What is a binary heap?

A binary heap is a complete binary tree that satisfies the heap property. In a max-heap, the key of each internal node is greater than or equal to the keys of its children. In a min-heap, the key of each internal node is less than or equal to the keys of its children. Binary heaps are used to implement priority queues and sorting algorithms like heap sort.

What is a binary tree traversal?

Binary tree traversal is the process of visiting each node in a binary tree exactly once. There are three main types of traversal: pre-order, in-order, and post-order. Pre-order traversal visits the root node first, followed by the left child, and then the right child. In-order traversal visits the left child first, then the root node, and then the right child. Post-order traversal visits the left child first, then the right child, and then the root node.

What is a binary tree application?

Binary trees have many applications in computer science. They are used in data structures for storing and organizing data, in algorithms for searching and sorting, and in graph algorithms for finding shortest paths and minimum spanning trees. Binary trees are also used in database systems for indexing and querying data.

What is a binary tree complexity?

The complexity of binary tree operations depends on the type of operation and the structure of the tree. For a balanced binary tree, the time complexity for searching, insertion, and deletion is $O(\log n)$, where n is the number of nodes. For an unbalanced tree, the complexity can be as high as $O(n)$.

What is a binary tree implementation?

There are two main ways to implement a binary tree: using arrays and using pointers. In the array-based implementation, nodes are stored in an array, and the left and right children of a node are calculated using simple arithmetic. In the pointer-based implementation, each node is a struct or object that contains a key and pointers to its left and right children.

What is a binary tree visualization?

Binary tree visualization is the process of drawing a binary tree on a 2D plane. There are several algorithms for visualizing binary trees, including breadth-first search (BFS) and depth-first search (DFS). Visualization helps in understanding the structure and properties of a binary tree.

What is a binary tree optimization?

Binary tree optimization involves techniques to improve the performance of binary tree operations. This can include balancing the tree to maintain $O(\log n)$ complexity, using lazy evaluation to avoid unnecessary calculations, and using memory-efficient data structures to store the tree nodes.

What is a binary tree extension?

Binary tree extensions are variations of the basic binary tree structure. Examples include B-trees (used in databases), AVL trees (self-balancing binary search trees), and segment trees (used in range queries). Each extension has its own set of properties and applications.